

Lab 7 Report 110550022

Introduction

This report goes over the implementation details and results of Lab 7. Through this assignment I found that normalization plays an important role in RL training, and the repercussions of choosing the wrong activation functions.

Implementation

How do you obtain the stochastic policy gradient and the TD error for A2C?

```
values = self.critic(states)
next_values = self.critic(next_states)

targets = rewards + self.gamma * next_values * (1 - dones)
value_loss = F.mse_loss(values, targets.detach())

advantages = (targets - values).detach()
entropy_loss = entropies.mean()

policy_loss = (
    -log_probs * advantages
).mean() - self.entropy_weight * entropy_loss
```

$$\frac{1}{|B|} \sum_{(s,a,r,s') \in B} \log \pi_{\theta}(a|s) \hat{A}_w(s, a),$$

The actor objective is the mean of log probabilities multiplied by the advantage (td error). This term is transformed into a loss by taking the negative. The entropy of the distribution is also added to promote exploration.

$$\frac{1}{|B|} \sum_{(s,a,r,s')} (r + \gamma V_w(s') - V_w(s))^2.$$

The value loss is the mean squared error between the td error and the values

How do you implement the clipped objective in PP0?

```
pg_loss1 = -adv * ratio
pg_loss2 = -adv * torch.clamp(ratio, 1 - self.epsilon, 1 + self.epsilon)
entropy_loss = dist.entropy().mean()

actor_loss = (
    torch.max(pg_loss1, pg_loss2).mean()
    - self.entropy_weight * entropy_loss
)
```

$$\frac{1}{|B|} \sum_{(s,a,r,s') \in B} \min(\rho_{s,a}(\theta) A^{\text{GAE}}(s,a), \text{clip}(\rho_{s,a}(\theta), 1 - \epsilon, 1 + \epsilon) A^{\text{GAE}}(s,a))$$

The objective function is the minimum of the probability ratio * advantage and its clipped variant. Because its negative is taken as the loss, max() is used instead of min().

How do you obtain the estimator of GAE?

$$\begin{aligned} \delta_t &= r_t + \gamma V(s_{t+1}) - V(s_t) \\ \hat{A}_t &= \delta_t + \gamma \lambda \delta_{t+1} + \dots + (\gamma \lambda)^{T-t+1} \delta_{T-1} \\ &= \delta_t + \gamma \lambda \hat{A}_{t+1} \end{aligned}$$

The recurrence relation function of GAE listed in [Annotated Research Paper](#)
[Implementations](#)

```
##### 1000 #####
values = values + [next_value]
gae = 0
gae_returns = []

for step in reversed(range(len(rewards))):
    delta = rewards[step] + gamma * values[step + 1] * masks[step] - values[step]
    gae = delta + gamma * tau * masks[step] * gae
    gae_returns.insert(0, gae + values[step])
#####
return gae_returns
```

Values is first extended by appending next_value to the last element, and the lists are iterated in reverse order.

The rest is just to implement the recurrence function:

$$\Delta = \text{reward} + \gamma * \text{value}_{t+1} - \text{value}_t$$

if the state of the environment terminates at time t, the value of t + 1 should be zeroed out. So, value_{t+1} is multiplied by the mask at time t.

$$\text{Gae}_t = \Delta + \gamma * \lambda * \text{gae}_{t+1}$$

and since the iteration is in reverse, the above is simply

$$\text{Gae} = \Delta + \gamma * \lambda * \text{gae}$$

The computed gae is inserted to the front of gae_returns so it's in the correct order.

How do you collect samples from the environment?

This section is implemented by the TAs already, so I'll only go over the method and not the code.

At each rollout step, the actor network predicts a probability distribution over actions based on the current observation. During training, an action is sampled from this distribution, and the environment transitions to the next state based on that action. The resulting transition is stored in a rollout buffer. Once the loop reaches a predetermined number of iterations, the agent performs multiple epochs of minibatch updates using the data from the buffer.

During testing, the agent takes the mean of the predicted action distribution instead of sampling to act deterministically

How do you enforce exploration (despite that both A2C and PPO are on-policy RL methods)?

Exploration is enforced through the output design of the actor network, where it outputs a probability distribution over actions. When sampling from this distribution, it inherently introduces randomness and enforces exploration

based on the variance of the predicted distribution.

Explain how you use Weight & Bias to track model performance and the loss values (including actor loss, critic loss, and entropy).

I've simply logged them using weights and biases, though the only metric I use to evaluate model performance is the reward as the losses in RL isn't as straightforward as in traditional deep learning tasks. Entropy loss, however, can show that the entropy parameter is really promoting exploration. When disabling entropy loss, the entropy generally declines throughout training, suggesting that the actor is becoming more and more deterministic as time goes on (less exploration). When entropy loss is on, entropy usually stabilizes to a non-zero value.

Analysis and Discussion

Plot the training curves

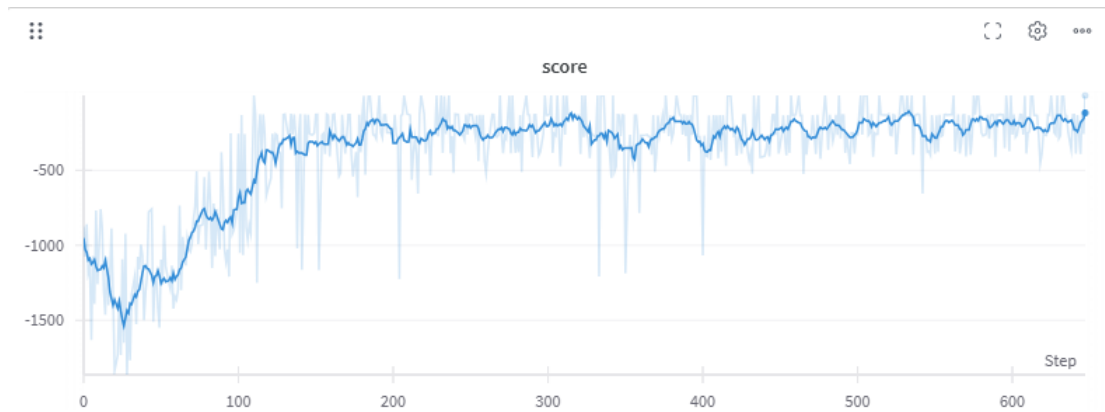
A2C Pendulum

Step is env_step / 16 since I used a mini batch size of 16 for a2c



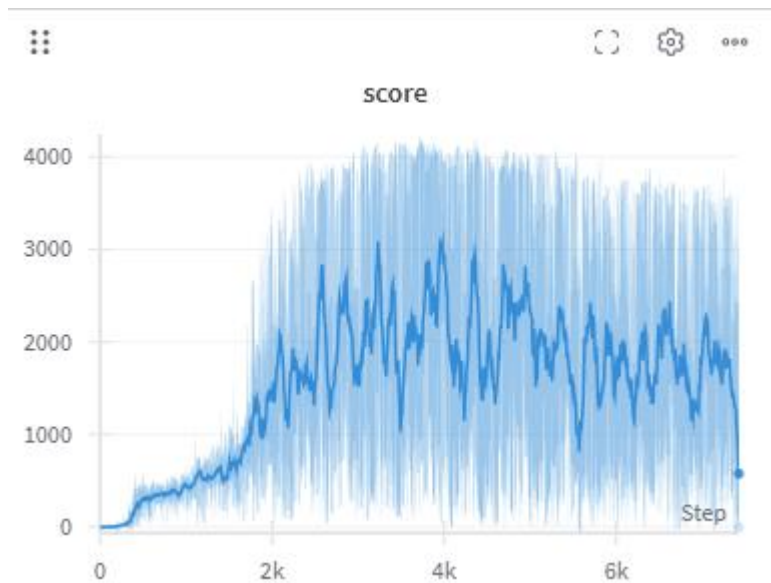
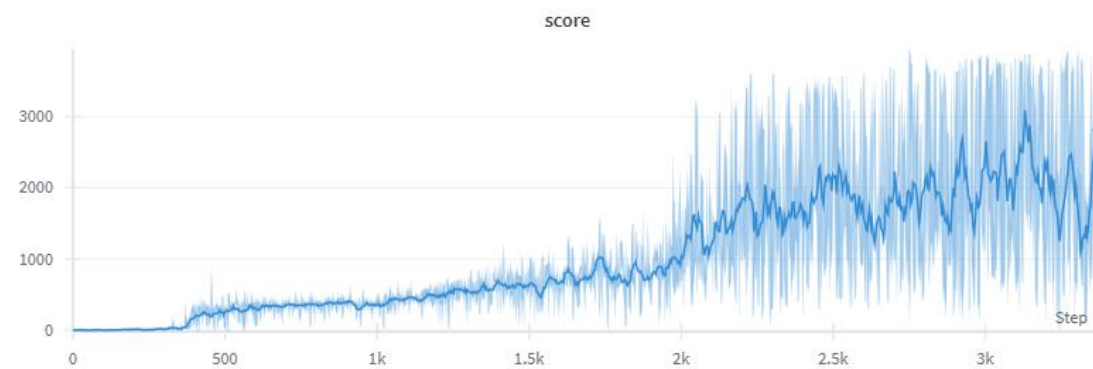
PPO Pendulum

Step of 600 corresponds to 100k env steps



PPO walker

Graph ends at 1mil steps



^Full 3Mil step graph. The model accuracy starts dropping after training too long for some reason. The snapshots were also performing worse at later steps, likely due to “Catastrophic Forgetting”.

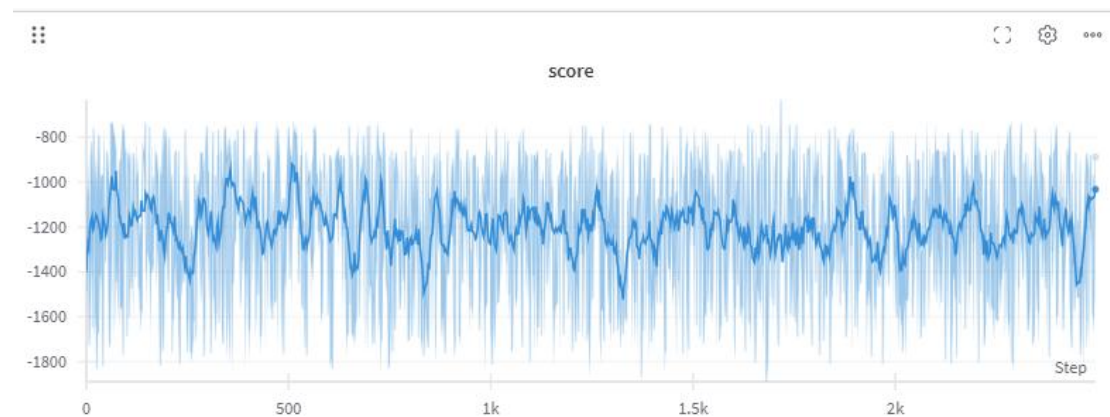
Compare the sample efficiency and training stability of A2C and PPO

PPO converges faster and is more stable early on. I also find that increasing mini batch size to 16 for a2c drastically improves training stability, though it's still quite unstable in the starting episodes.

Perform an empirical study on the key parameters, such as clipping parameter and entropy coefficient

Clipping parameter

Experimentations were conducted on the ppo-pendulum model and I found that without clipping the PPO model doesn't converge.



Entropy coefficient:

Entropy is most noticeable on A2C as it speeds up convergence by a lot, but not so much on PPO methods. I also don't see a lot of reward gain from incorporating entropy, which matches the experiments conducted by Andrychowicz et al. (in WHAT MATTERS FOR ON-POLICY DEEP ACTORCRITIC METHODS? A LARGE-SCALE STUDY)

Additional Analysis on other strategies

Most of the strategies are referenced from the ICLR blog post

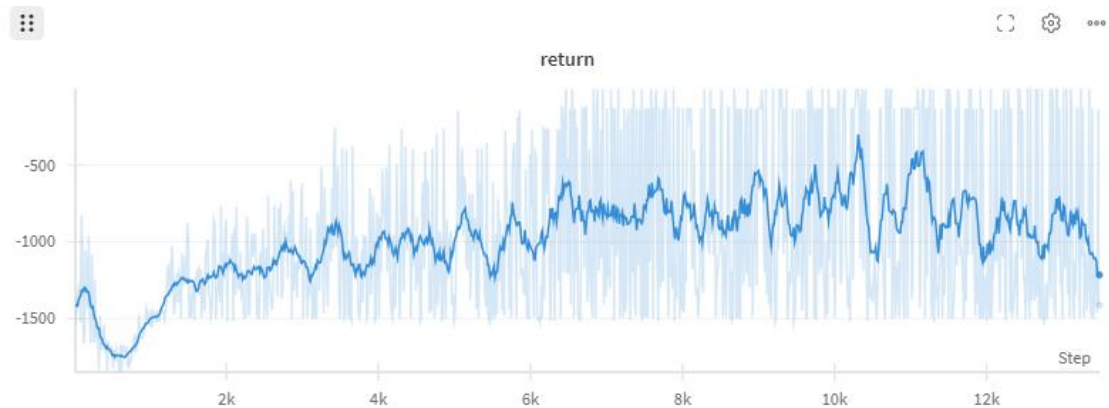
<https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/>

Global

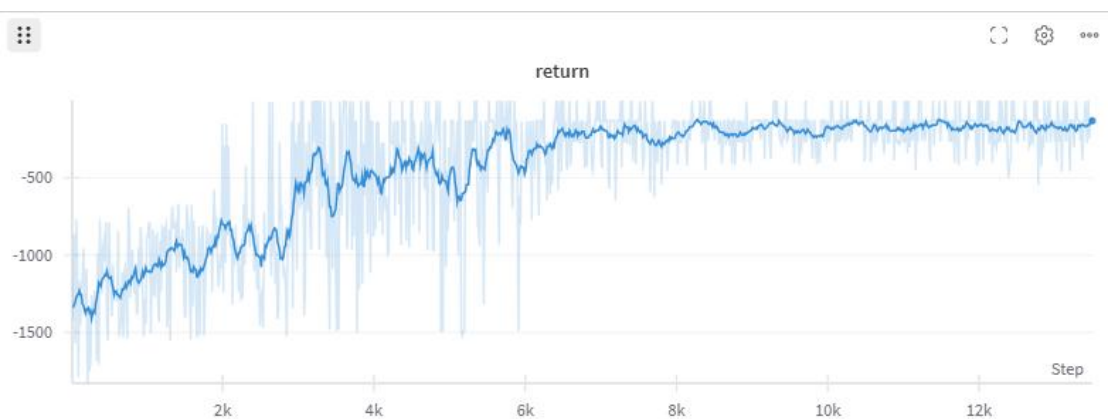
Observation Normalization

This is the single biggest improvement to model training stability and performance, and it simply involves wrapping the environment in `gym.wrappers.NormalizeObservation()` and saving the running mean, variance and count alongside the models.

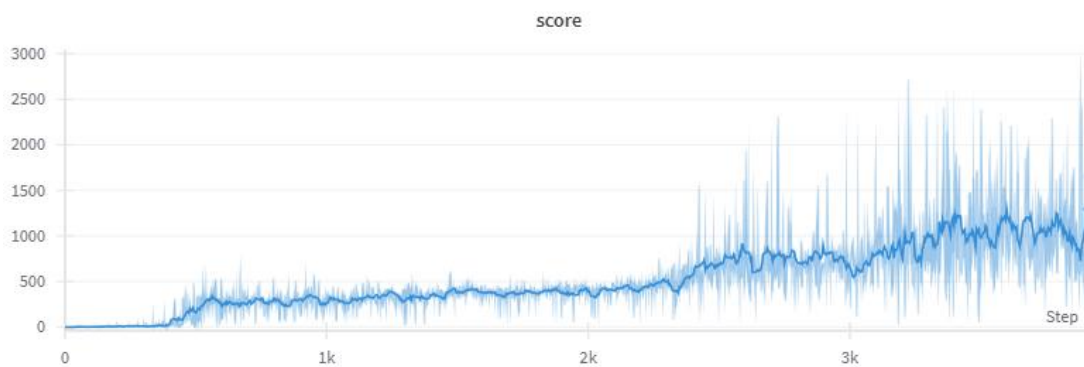
A2C with no normalization



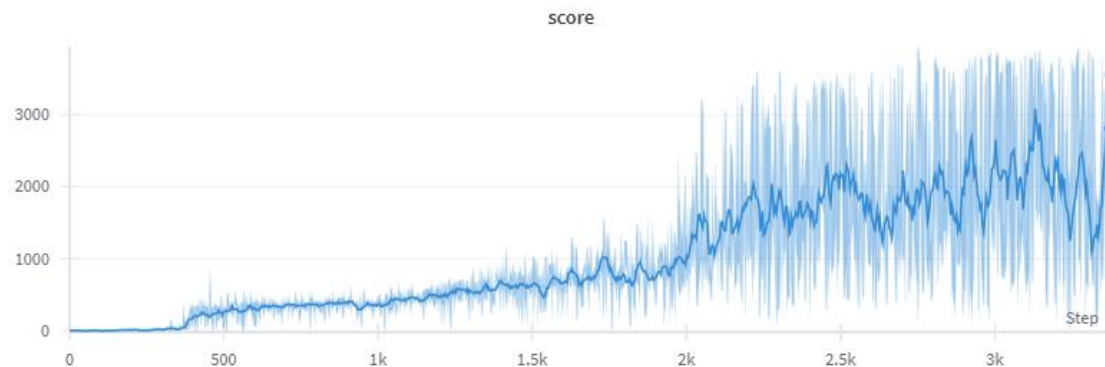
A2C with normalization



Walker without normalization



Walker with normalization



A2C

Activation function

I found that using the exact same initialization and network architectures from PPO (see **Orthogonal Initialization of Weights and Constant Initialization of biases** in the next section) causes A2C to not converge at all. After switching to SiLU or Mish and using the default kaiming initialization, the model was able to converge nicely.

Mini batch size

From my testing, the default mini batch size of 1 is too small and causes significant training instabilities. Thus, the training loop is rewritten in a fashion similar to PPO's training loop to implement non-one mini batch sizes (16 works decently well).

With the above modifications, A2C was able to match the stability of PPO in the later epochs.

PPO

PPO_iter rewrite

The mini-batch update section in the blogpost suggests that randomly sampling batches from the training data is a common mis-implementation. So I've rewritten the ppo_iter function to use shuffling and slicing as suggested.

Sum of log probs

According to the ppo_continuous_action.py file in the github repository linked in the blog post and pretty much every other continuous action space PPO implementation, the log probabilities and entropy should be summed along the action dimension ([64, 6] -> [64,]). I have reflected that in my implementation. This was not done on the pendulum model since the dimension is 1 and summing essentially does nothing.

Incorrect handling of terminal states

I mentioned in my DQN report that truncation and termination should be handled differently, so I thought this would also be the case for PPO. However, the blog post did not differentiate them to correctly reproduce the results, and the reward they obtained is good, so I decided to just not bother with it. (point 5,

A note on truncation)

Normalization of advantages

I can't really see tell much of a difference, which aligns with the reports by [Andrychowicz, et al.](#), but this is specifically for PPO. I find that normalizing advantages hurt A2C's performance.

Orthogonal Initialization of Weights and Constant Initialization of biases

I did not do an ablation study on this, but most papers reports that this is beneficial, so I've replaced the uniform normalization in the template with the suggested configurations based on the blog post.

With the above modifications, the PPO model was able to learn Walker2d with a peak training reward of 4k and moving average of close to 3k. During inference, the model was able to generate 4k scores quite frequently.